ML Supervised Techniques

Master Summary

Stefan Eder

Based on Lecture *Supervised Learning* SS25

This document is a compact summary of the course. It aims to cover all relevant concepts and results, but long derivations, detailed proofs, and worked examples are often condensed or omitted in favour of clarity and conciseness. Use it as a study aid alongside the lecture notes and slides, not as a replacement for them.

If you spot an error or a gap, please open an issue or pull request at **github.com/stevetgit** — corrections are always welcome.

Good study materials should be shared, not gatekept. If this helped you, take it as a reason to help the next colleague you see struggling with something.

March 14, 2026

Contents

1 Introduction & Basics **1**

1.1 Supervised learning setup and notation 1

1.2 Loss, generalization error (risk), and empirical risk 1

1.3 Generalization, underfitting, overfitting, and model complexity 2

1.4 Train/validation/test and cross-validation 2

1.4.1 Cross-validation: calculation recipe (incl. a simple threshold model) 2

1.5 Probabilistic framework and Bayes optimal classifier 3

1.6 Intro models: kNN and regression 3

1.6.1 k-Nearest Neighbors (kNN) 3

1.6.2 Linear and polynomial regression 3

1.7 Evaluation: confusion matrix, ROC/AUC, PR curves 3

1.7.1 Confusion matrix and common metrics 3

1.7.2 ROC curve and AUC 4

1.7.3 Precision–Recall (PR) curves 4

1.8 Bias–variance decomposition (high-level) 4

2 Support Vector Machines **5**

2.1 Geometric view: hyperplanes and margin 5

2.1.1 Convex-hull perspective (2D exam geometry) 5

2.2 Hard-margin SVM (linearly separable case) 5

2.3 KKT conditions and support vectors 6

2.3.1 KKT & Lagrange duality: template for exam mini-problems 6

2.4 Soft-margin SVM (C-SVM) and hinge loss 7

2.5 Kernel trick (nonlinear SVMs) 7

2.6 Multi-class SVMs and SVR (outline-level) 7

2.6.1 Multi-class 7

2.6.2 Support Vector Regression (SVR) 8

2.7 Pros/cons (what to remember) 8

3 Random Forests and Gradient Boosting **9**

3.1 Decision trees: basic concepts 9

3.1.1 What a decision tree represents 9

3.1.2 Recursive learning and the three design issues 9

3.1.3 Splitting criteria for classification 9

3.1.4 Splitting criteria for regression 10

3.1.5 Overfitting and regularization in trees 10

3.2 Ensembles: bagging vs. boosting (one sentence each) 10

3.3 Random forests 10

3.3.1 Algorithm 10

3.3.2 Why it works: variance reduction and decorrelation 10

3.3.3 Out-of-bag (OOB) error and variable importance 10

3.3.4 Class imbalance 11

3.3.5 Pros/cons (typical exam talking points) 11

3.4 Boosting . . . 11
3.4.1 Forward stagewise additive modeling (general template) . . . 11
3.4.2 AdaBoost (binary classification intuition) . . . 11
3.4.3 Gradient boosting and tree boosting . . . 12
3.4.4 XGBoost (high level) . . . 13
3.5 RF vs. GBT: quick comparison . . . 13

4 Logistic Regression . . . 14
4.1 Maximum likelihood estimation (MLE) . . . 14
4.1.1 Likelihood and log-likelihood . . . 14
4.1.2 Gaussian noise implies squared error (important equivalence) . . . 14
4.2 Binary logistic regression . . . 14
4.2.1 Model and sigmoid . . . 14
4.2.2 Bernoulli likelihood . . . 14
4.2.3 Cross-entropy loss (negative log-likelihood) . . . 15
4.2.4 Gradients and convexity (what exams usually target) . . . 15
4.3 Optimization in practice . . . 15
4.3.1 Gradient descent and learning rate . . . 15
4.3.2 Momentum . . . 15
4.3.3 Gradient checking (debugging tool) . . . 15
4.4 Multi-class logistic regression (softmax regression) . . . 16

5 Artificial Neural Networks . . . 17
5.1 Perceptron and linear separability . . . 17
5.1.1 Perceptron as a linear threshold unit . . . 17
5.1.2 Perceptron learning algorithm (online updates) . . . 17
5.1.3 Linear separability, convergence, and XOR limitation . . . 17
5.2 Feed-forward neural networks (MLPs / FNNs) . . . 18
5.2.1 Architecture and forward pass notation . . . 18
5.2.2 Activation functions: sigmoid/tanh vs. ReLU/ELU . . . 18
5.2.3 Loss functions (link to probabilistic assumptions) . . . 18
5.2.4 Universal Approximation Theorem (exam-level statement) . . . 18
5.3 Training: empirical risk, gradient descent, backpropagation . . . 18
5.3.1 Empirical risk and gradient descent update . . . 19
5.3.2 Backpropagation via delta errors and layer Jacobians . . . 19
5.3.3 Delta error at the output layer (three important special cases) . . . 20
5.3.4 Backprop procedure (what you actually do) . . . 20
5.4 Vanishing/exploding gradients (deep nets) . . . 20
5.5 Deep learning tricks: initialization and regularization . . . 21
5.5.1 Weight initialization (keep signal/gradients stable) . . . 21
5.5.2 Dropout (prevent co-adaptation) . . . 21
5.5.3 Batch normalization (BN) (stabilize learning) . . . 21
5.5.4 Weight decay (L2 regularization) . . . 22
5.5.5 Early stopping (validation-based regularization) . . . 22
5.6 Double descent (optional intuition) . . . 22

6 Special Network Architectures . . . 23
6.1 Convolutional Neural Networks (CNNs) . . . 23
6.1.1 Why CNNs: image structure and local receptive fields . . . 23
6.1.2 Convolution vs. cross-correlation and weight sharing . . . 23
6.1.3 Stride and padding . . . 23

6.1.4 Pooling (subsampling) 24
6.1.5 Inputs and outputs: channels and multiple kernels 24
6.1.6 Example architectures and transfer learning (high level) 24
6.1.7 Dataset augmentation (exam-relevant) 24
6.2 Recurrent Neural Networks (RNNs) 24
6.2.1 Sequence learning settings 25
6.2.2 RNN forward pass and weight sharing across time 25
6.2.3 Training: unrolling and Backpropagation Through Time (BPTT) 25
6.2.4 Vanishing/exploding gradients in RNNs 25
6.2.5 Long Short-Term Memory (LSTM): idea 25
6.2.6 Limitations and modern alternatives (very high level) 26
6.3 Attention and transformers (exam-level facts) 26

1

Introduction & Basics

1.1 Supervised learning setup and notation

Supervised learning uses labeled data to learn a predictive model. We observe a dataset

$$Z = \{(x_i, y_i)\}_{i=1}^{l}, \quad x_i \in \mathcal{X} \subseteq \mathbb{R}^d, \quad y_i \in \mathcal{Y},$$

and choose a model class (hypothesis class) of functions

$$g(\cdot; w) : \mathcal{X} \to \hat{\mathcal{Y}},$$

parametrized by weights/parameters w.

Typical tasks:

- **Classification:** $\mathcal{Y}$ is discrete (binary or multi-class). Often $\hat{\mathcal{Y}} = \mathcal{Y}$ or predicted probabilities.
- **Regression:** $\mathcal{Y} \subseteq \mathbb{R}$ (or $\mathbb{R}^m$); predict real-valued targets.

1.2 Loss, generalization error (risk), and empirical risk

We measure prediction quality with a loss function $L(y, \hat{y})$. The central quantity is the **generalization error** (also called **risk**):

$$R(w) = \mathbb{E}_{(x,y)\sim p(x,y)}\Big[L(y, g(x; w))\Big].$$

Since $p(x, y)$ is unknown, we approximate it from samples.

Empirical Risk Minimization (ERM)

Given $Z = \{(x_i, y_i)\}_{i=1}^{l}$,

$$R_{\text{emp}}(w) = \frac{1}{l}\sum_{i=1}^{l} L(y_i, g(x_i; w)), \qquad \hat{w} \in \arg\min_{w} R_{\text{emp}}(w).$$

Key assumption for using sample averages to estimate expectations: training and test data are (approximately) **i.i.d.** from the same distribution. In practice, violations (dataset shift, leakage, time dependence) are a major source of failure.

1.3 Generalization, underfitting, overfitting, and model complexity

ERM alone can produce models that fit noise. A standard diagnostic is the comparison of training and test performance:

- **Underfitting:** training error high; model class too simple (high bias).
- **Overfitting:** training error low but test error high; model too flexible (high variance).

A useful mental model is the **bias–variance trade-off**: increasing model complexity typically *decreases bias* but *increases variance*. The goal is to minimize the generalization error, not the training error.

1.4 Train/validation/test and cross-validation

A standard workflow:

- **Training set:** fit parameters w.
- **Validation set:** choose hyperparameters (model class complexity, regularization strength, k in kNN, C in SVM, depth in trees, ...).
- **Test set:** used once to estimate generalization error; must not be used for model selection, feature selection, or preprocessing decisions.

For small datasets, cross-validation (CV) reuses data efficiently. In K-fold CV, split Z into K disjoint folds; train on $K-1$ folds and evaluate on the left-out fold; average the K test errors to obtain a more stable estimate.

1.4.1 Cross-validation: calculation recipe (incl. a simple threshold model)

Some past exams include a very explicit CV calculation: for each fold k you train on the union of all other folds and evaluate on the held-out fold F_k. The CV risk is the average over folds:

$$L_{\text{cv}} = \frac{1}{K}\sum_{k=1}^{K} L_{\text{cv},k}, \qquad L_{\text{cv},k} = \frac{1}{|F_k|}\sum_{(x_i,y_i)\in F_k} L\big(y_i, \hat{y}_i^{(-k)}\big),$$

where $\hat{y}_i^{(-k)}$ is the prediction from the model trained without fold k.

Exam: Threshold Model

Sometimes the model is fixed as

$$g(x,w) = \begin{cases} +1, & x \geq w, \\ -1, & \text{otherwise}, \end{cases}$$

and the parameter is (unusual but exam-relevant) computed on the training set via

$$w = \frac{1}{l}\sum_{i=1}^{l} |x_i - y_i|.$$

Then one evaluates on the held-out fold with 0-1 loss $L_{01}(y,\hat{y}) = \mathbb{1}[y \neq \hat{y}]$.

1.5 Probabilistic framework and Bayes optimal classifier

In probabilistic classification we model (or reason about) $p(y \mid x)$. With **zero-one loss** $L_{01}(y, \hat{y}) = \mathbb{1}[y \neq \hat{y}]$, the Bayes optimal classifier is

$$g^*(x) \in \arg\max_{y \in \mathcal{Y}} p(y \mid x),$$

i.e. predict the most probable class given x.

Bayes' theorem connects conditional and joint distributions:

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{p(x)}, \qquad p(x) = \sum_{y \in \mathcal{Y}} p(x \mid y)p(y)$$

(or integral over y if continuous).

1.6 Intro models: kNN and regression

1.6.1 k-Nearest Neighbors (kNN)

kNN is a non-parametric method: for a new point x_0, find its k closest training samples (by a distance, e.g. Euclidean) and predict:

- **Classification:** majority vote among the k labels (possibly distance-weighted).
- **Regression:** average of the k target values.

k controls complexity: small k can overfit; large k can underfit.

1.6.2 Linear and polynomial regression

For regression with quadratic loss, typical models are

$$g(x; w) = w_0 + w_1 x \quad \text{(linear in 1D)}, \qquad g(x; w) = \sum_{j=0}^{m} w_j x^j \quad \text{(polynomial in 1D)}.$$

Quadratic loss (mean squared error):

$$L_q(y, g) = (y - g)^2, \qquad R_{\text{emp}}(w) = \frac{1}{l} \sum_{i=1}^{l} \big(y_i - g(x_i; w)\big)^2.$$

Increasing polynomial degree m increases model flexibility; the best generalization typically occurs at an intermediate degree.

1.7 Evaluation: confusion matrix, ROC/AUC, PR curves

1.7.1 Confusion matrix and common metrics

For binary classification, define counts:

$$\text{TP, FP, TN, FN.}$$

Common derived metrics:

$$\text{Accuracy} = \frac{\text{TP}+\text{TN}}{\text{TP}+\text{FP}+\text{TN}+\text{FN}}, \quad \text{TPR/Recall} = \frac{\text{TP}}{\text{TP}+\text{FN}}, \quad \text{TNR/Specificity} = \frac{\text{TN}}{\text{TN}+\text{FP}},$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP}+\text{FP}}, \quad \text{Balanced Acc} = \frac{1}{2}(\text{TPR}+\text{TNR}), \quad F_1 = \frac{2\,\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}}.$$

Balanced accuracy is often preferred for **unbalanced** datasets.

1.7.2 ROC curve and AUC

A score-based classifier yields a real-valued discriminant $\bar{g}(x)$ and predicts class $+1$ if $\bar{g}(x) \geq \tau$. Varying τ produces different (FPR, TPR) points:

$$\text{FPR} = \frac{\text{FP}}{\text{FP}+\text{TN}}, \qquad \text{TPR} = \frac{\text{TP}}{\text{TP}+\text{FN}}.$$

The ROC curve plots TPR vs. FPR over thresholds. The AUC is the area under the ROC curve (can be computed by trapezoids after sorting thresholds).

ROC Point at Fixed Threshold ϕ

Given scores $\bar{g}(x)$ and a threshold rule

$$\hat{y} = \begin{cases} +1, & \bar{g}(x) \geq \phi, \\ -1, & \text{otherwise}, \end{cases}$$

compute the confusion matrix once for this ϕ and then

$$\text{FPR} = \frac{\text{FP}}{\text{FP}+\text{TN}}, \qquad \text{TPR} = \frac{\text{TP}}{\text{TP}+\text{FN}}.$$

This single point (FPR, TPR) is exactly the location on the ROC curve corresponding to ϕ.

1.7.3 Precision–Recall (PR) curves

PR curves plot Precision vs. Recall over thresholds and are particularly informative under strong class imbalance.

1.8 Bias–variance decomposition (high-level)

For regression with squared error, a standard decomposition at a fixed point x_0 is:

$$\mathbb{E}\big[(y - g(x_0; w(Z)))^2\big] = \underbrace{\text{Var}[y \mid x_0]}_{\text{noise}} + \underbrace{\big(\mathbb{E}[g(x_0; w(Z))] - \mathbb{E}[y \mid x_0]\big)^2}_{\text{bias}^2} + \underbrace{\text{Var}\big[g(x_0; w(Z))\big]}_{\text{variance}}.$$

In binary classification with labels in $\{-1, +1\}$ and predictions restricted to $\{-1, +1\}$, the zero-one loss can be expressed via a squared difference (up to a constant factor), which allows analogous bias/variance reasoning.

2

Support Vector Machines

2.1 Geometric view: hyperplanes and margin

Binary labels are typically $y_i \in \{-1, +1\}$ and $x_i \in \mathbb{R}^d$. A hyperplane is given by

$$\langle w,\, x\rangle - b = 0,$$

with normal vector w and offset b. The classifier predicts

$$g(x) = \operatorname{sign}(\langle w,\, x\rangle - b).$$

For linearly separable data, we can scale (w, b) so that

$$y_i(\langle w,\, x_i\rangle - b) \geq 1 \quad \forall i.$$

Then the margin (distance between the two supporting hyperplanes) equals $2/\,\|w\|$, so maximizing margin is equivalent to minimizing $\|w\|$.

2.1.1 Convex-hull perspective (2D exam geometry)

A dataset is linearly separable iff the convex hulls of the two classes do not intersect. In well-separated 2D problems, the maximum-margin separator is geometrically "between" the two hulls.

Line from closest hull pair (useful heuristic in exams): If you identify two closest points p_+ (class $+1$) and p_- (class -1) on the hulls, then a max-margin boundary is often the line through the midpoint $m = \frac{1}{2}(p_+ + p_-)$ with normal vector proportional to $(p_- - p_+)$:

$$w \propto (p_- - p_+), \qquad w^\top x - b = 0, \qquad b = w^\top m.$$

To convert to $x_2 = kx_1 + d$ (in 2D, $w = (w_1, w_2)$):

$$w_1x_1 + w_2x_2 - b = 0 \quad \Longrightarrow \quad x_2 = \underbrace{\left(-\frac{w_1}{w_2}\right)}_{k} x_1 + \underbrace{\left(\frac{b}{w_2}\right)}_{d}.$$

Area of a convex hull polygon (shoelace formula): For ordered vertices $(x_1, y_1), \ldots, (x_n, y_n)$,

$$\text{Area} = \frac{1}{2}\left|\sum_{i=1}^{n} x_i y_{i+1} - y_i x_{i+1}\right|, \quad (x_{n+1}, y_{n+1}) = (x_1, y_1).$$

2.2 Hard-margin SVM (linearly separable case)

Primal Hard-Margin SVM

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(\langle w, x_i\rangle - b) \geq 1, \quad i = 1, \ldots, l.$$

The associated dual problem yields Lagrange multipliers $\alpha_i \geq 0$. The solution has the form

$$w = \sum_{i=1}^{l} \alpha_i y_i x_i,$$

and only points with $\alpha_i > 0$ influence w.

2.3 KKT conditions and support vectors

KKT complementarity implies

$$\alpha_i(y_i(\langle w, x_i\rangle - b) - 1) = 0.$$

Hence, if $\alpha_i > 0$, then $y_i(\langle w, x_i\rangle - b) = 1$ and x_i lies on the margin boundary: it is a **support vector**.

For any support vector x_j one can compute b via

$$b = -y_j + \langle w, x_j\rangle,$$

and in practice one averages b over all support vectors for numerical stability.

2.3.1 KKT & Lagrange duality: template for exam mini-problems

Many constrained problems can be written as

$$\min_w f(w) \quad \text{s.t.} \quad h(w) \leq 0.$$

The Lagrangian is

$$\mathcal{L}(w, \lambda) = f(w) + \lambda\, h(w), \qquad \lambda \geq 0.$$

KKT conditions:

- Primal feasibility: $h(w^*) \leq 0$
- Dual feasibility: $\lambda^* \geq 0$
- Stationarity: $\nabla_w \mathcal{L}(w^*, \lambda^*) = 0$
- Complementary slackness: $\lambda^* h(w^*) = 0$

KKT: Constrained Minimization onto Hyperplane

$\min_w \frac{1}{2}\|w\|^2$ s.t. $a^\top w \leq b$.
Unconstrained optimum is $w = 0$. If $a^\top 0 \leq b$ then $w^* = 0$.
If $a^\top 0 > b$, the constraint is active ($a^\top w = b$) and the minimizer is the projection onto the hyperplane:

$$w^* = \frac{b}{\|a\|^2}\, a.$$

2.4 Soft-margin SVM (C-SVM) and hinge loss

If data are not separable, introduce slack variables $\xi_i \geq 0$:

$$y_i(\langle w,\, x_i\rangle - b) \geq 1 - \xi_i.$$

The soft-margin objective trades off margin size and violations:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{l}\xi_i,$$

with $C > 0$ controlling the trade-off. This corresponds to a margin-based classification with the hinge loss.

Soft-Margin: Dual Bounds and KKT Cases

In the soft-margin case one typically has

$$0 \leq \alpha_i \leq C$$

(and additionally $\sum_i \alpha_i y_i = 0$ in the standard dual).

KKT case distinction (memorize):

- $\alpha_i = 0$: point is not a support vector (correct and outside the margin).
- $0 < \alpha_i < C$: point lies on the margin (a "margin SV"), typically $\xi_i = 0$.
- $\alpha_i = C$: point violates the margin ($\xi_i > 0$); if $\xi_i > 1$ it is misclassified.

This is useful for plot/geometry questions where you must identify or count support vectors.

2.5 Kernel trick (nonlinear SVMs)

Replace the dot product by a kernel

$$K(x, z) = \langle \phi(x),\, \phi(z)\rangle\,,$$

without explicitly constructing ϕ. The decision function becomes

$$g(x) = \operatorname{sign}\left(\sum_{i=1}^{l}\alpha_i y_i K(x_i, x) - b\right).$$

Typical kernels: linear, polynomial, Gaussian/RBF.

A very common special case is the Gaussian/RBF kernel:

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right) = \exp\left(-\gamma\,\|x - z\|^2\right), \qquad \gamma = \frac{1}{2\sigma^2}.$$

2.6 Multi-class SVMs and SVR (outline-level)

2.6.1 Multi-class

Common approaches: one-vs-rest or one-vs-one; decision by maximum score / voting.

For M classes, two standard strategies are exam-relevant:

One-vs-Rest (OvR): train M binary SVMs. Classifier m separates class m vs. the rest. Decision is typically by maximum score:

$$\hat{y}(x) = \underset{m \in \{1,\dots,M\}}{\arg\max} \; f_m(x).$$

One-vs-One (OvO): train $\frac{M(M-1)}{2}$ binary SVMs (one for each pair m vs. n). Decision is usually by majority vote over all pairwise classifiers (or by aggregating pairwise scores).

2.6.2 Support Vector Regression (SVR)

SVR uses an ε-insensitive tube and slack variables (above/below the tube). The parameter C again controls the trade-off between flatness and training error.

Exam Pitfall: SVR Uses Real-Valued Output

SVR is *regression*. You use the real-valued output $f(x)$ (magnitude/offset matters), not only $\text{sign}(f(x))$. This matches the ε-insensitive tube around $f(x)$.

2.7 Pros/cons (what to remember)

- Pros: strong theoretical foundation; convex optimization $\Rightarrow$ global optimum; often effective when $d \gg l$.
- Cons: can be slow on very large datasets; sensitive to noise/overlap; no intrinsic probabilistic output.

3

Random Forests and Gradient Boosting

3.1 Decision trees: basic concepts

3.1.1 What a decision tree represents

A decision tree is a predictive model that maps an input x to an output by repeatedly applying decision rules: internal nodes test a feature condition, branches correspond to test outcomes, leaves output the final prediction.

- **Classification tree:** leaf outputs a class label (often via majority vote) and sometimes class probabilities.
- **Regression tree:** leaf outputs a real value (often the mean of training targets in the leaf).

3.1.2 Recursive learning and the three design issues

Decision tree learning is typically a recursive, depth-first procedure that splits the training set into subsets, aiming for leaf nodes that are homogeneous in the target variable.

Three key design issues:

1. **Splitting criterion:** which split to choose at a node?
2. **Stopping criterion:** when to stop growing the tree (controls complexity)?
3. **Pruning:** whether/how to collapse deep subtrees to avoid overfitting.

3.1.3 Splitting criteria for classification

Let Z be the data arriving at a node with class probabilities p_k for $k = 1, \ldots, M$.

Entropy:

$$H(Z) = -\sum_{k=1}^{M} p_k \ln p_k, \qquad H(Z) \leq \ln M \text{ (maximum at uniform classes).}$$

For a split s producing subsets $Z_{s,t}$, the **information gain** is

$$g_E(Z, s) = H(Z) - \sum_t \frac{|Z_{s,t}|}{|Z|} H(Z_{s,t}).$$

A split maximizes information gain when it produces (approximately) homogeneous subsets.

Gini impurity:

$$G(Z) = 1 - \sum_{k=1}^{M} p_k^2, \qquad G(Z) \le 1 - \frac{1}{M}.$$

Gini gain is defined analogously by impurity reduction after the split.

3.1.4 Splitting criteria for regression

A common criterion for regression trees is **variance reduction**. If Z contains targets y_i, define

$$\mathrm{Var}(Z) = \frac{1}{|Z|} \sum_{i \in Z} (y_i - \bar{y})^2, \quad \bar{y} = \frac{1}{|Z|} \sum_{i \in Z} y_i.$$

Then choose splits that reduce the (weighted) variance in children.

3.1.5 Overfitting and regularization in trees

Deep trees can overfit (low bias, high variance). Typical controls: max depth, minimum leaf size, minimum impurity decrease, and pruning (e.g. cost-complexity pruning).

3.2 Ensembles: bagging vs. boosting (one sentence each)

Bagging (bootstrap aggregating) reduces variance by averaging many high-variance models trained on resampled data. **Boosting** reduces error by building an additive model sequentially, focusing later models on earlier mistakes.

3.3 Random forests

3.3.1 Algorithm

A random forest is an ensemble of decision trees built with two sources of randomness:

1. **Bootstrap samples:** each tree is trained on a bootstrap subsample of the training set.
2. **Random feature subsets:** at each split, only a random subset of features is considered.

Prediction aggregates trees: majority vote (classification) or average (regression).

3.3.2 Why it works: variance reduction and decorrelation

Averaging reduces variance, but the reduction is limited if trees are highly correlated. Intuition: random feature selection reduces correlation, which strengthens variance reduction.

3.3.3 Out-of-bag (OOB) error and variable importance

OOB idea: for each training point, evaluate it only on those trees whose bootstrap sample did *not* include it (roughly $\approx 36\%$ left out per tree). This provides a nearly-free generalization estimate.

Variable importance is commonly measured by

- **Mean decrease in impurity** (e.g. total Gini decrease attributed to a feature),

- **Permutation importance** (mean decrease in accuracy when permuting a feature).

Be aware that importance can be biased (e.g. categorical variables with many levels).

3.3.4 Class imbalance

A practical strategy to improve balanced accuracy is to **stratify classes when sampling** (or use class weights), so minority samples are not drowned out.

3.3.5 Pros/cons (typical exam talking points)

RFs are usually easy to tune (often mainly number of trees + features per split) and relatively robust to noise. Downsides include slower prediction with many trees and potential biases in importance measures.

3.4 Boosting

3.4.1 Forward stagewise additive modeling (general template)

Boosting can be viewed as building an additive model

$$g_N(x) = \sum_{n=1}^{N} \beta_n \, b(x; \gamma_n),$$

by repeatedly adding one new basis function.

Forward Stagewise Additive Modeling

Initialize $g_0(x) = 0$. For $n = 1, \ldots, N$:

$$(\beta_n, \gamma_n) = \arg\min_{\beta,\gamma} \sum_{i=1}^{l} L\big(y_i,\, g_{n-1}(x_i) + \beta \, b(x_i; \gamma)\big),$$

and update

$$g_n(x) = g_{n-1}(x) + \beta_n b(x; \gamma_n).$$

AdaBoost.M1 can be derived as a special case with exponential loss; the key point is that **coefficients from earlier steps are not re-optimized**.

3.4.2 AdaBoost (binary classification intuition)

AdaBoost maintains weights on data points. After fitting a weak classifier, **misclassified points get increased weight** and thus receive more attention in the next iteration. More accurate weak classifiers receive larger influence in the final vote.

A common final prediction form is:

$$g(x) = \operatorname{sign}\left(\sum_{n=1}^{N} \varrho_n \, g_n(x) \right),$$

where ϱ_n is the model weight (larger when the weak classifier error is smaller).

AdaBoost: Binary Standard Formulas

With data weights $D_t(i)$ and weak classifier $h_t(x) \in \{-1, +1\}$:

$$\epsilon_t = \sum_{i=1}^{l} D_t(i)\, \mathbb{1}[h_t(x_i) \neq y_i], \qquad \alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}.$$

Update the data weights:

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t},$$

where Z_t normalizes so $\sum_i D_{t+1}(i) = 1$.
Final classifier:

$$H(x) = \operatorname{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right).$$

AdaBoost Weight Update Intuition

Misclassified points have $y_i h_t(x_i) = -1$ and get multiplied by $\exp(+\alpha_t)$ (upweighted), correctly classified points get multiplied by $\exp(-\alpha_t)$ (downweighted).
Who gets the larger model weight? The weak learner with smaller error ϵ_t gets a larger α_t.

3.4.3 Gradient boosting and tree boosting

For trees, write a regression/classification tree as

$$T(x; \Theta) = \sum_{j=1}^{J} \gamma_j\, \mathbb{1}[x \in R_j], \quad \Theta = \{(R_j, \gamma_j)\}_{j=1}^{J}.$$

A boosted tree model is an additive sum of trees:

$$g_N(x) = \sum_{n=1}^{N} T(x; \Theta_n).$$

Gradient boosting views optimization in function space: let $g = (g(x_1), \ldots, g(x_l)) \in \mathbb{R}^l$ and

$$\mathcal{L}(g) = \sum_{i=1}^{l} L(y_i, g(x_i)).$$

At iteration n, compute the gradient of $\mathcal{L}$ w.r.t. the current predictions and move in the steepest descent direction. Because the negative gradient is unconstrained, we **fit a tree to approximate the negative gradient** (often called pseudo-residuals).

Practical hyperparameters for gradient boosted trees (GBTs):

- number of trees N,
- tree depth (often shallow),
- learning rate (shrinkage).

GBTs can overfit noisy data more easily than RFs because trees are built sequentially and later trees adapt to earlier errors.

3.4.4 XGBoost (high level)

XGBoost is a gradient-boosted tree method with explicit regularization (via complexity penalties) and many engineering improvements for speed/robustness. It is still widely considered highly competitive on tabular data.

3.5 RF vs. GBT: quick comparison

- RF: parallel training, robust, easy tuning, strong baseline.
- GBT: sequential training, very flexible loss functions, often higher accuracy but more sensitive and harder to tune.

4

Logistic Regression

4.1 Maximum likelihood estimation (MLE)

4.1.1 Likelihood and log-likelihood

Assume a probabilistic model $p(y \mid x; \omega)$ with parameters ω. Given i.i.d. data, the likelihood is a product:

$$\mathcal{L}(\omega) = \prod_{i=1}^{l} p(y_i \mid x_i; \omega),$$

and the log-likelihood is a sum:

$$\log \mathcal{L}(\omega) = \sum_{i=1}^{l} \log p(y_i \mid x_i; \omega).$$

MLE chooses $\hat{\omega} \in \arg\max_{\omega} \log \mathcal{L}(\omega)$. Equivalently, one minimizes the **negative log-likelihood** (NLL).

4.1.2 Gaussian noise implies squared error (important equivalence)

If a regression target is modeled as $y = g(x) + \varepsilon$ with Gaussian noise $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, then minimizing the NLL is equivalent (up to constants) to minimizing squared error.

4.2 Binary logistic regression

4.2.1 Model and sigmoid

Logistic regression models a probability for the positive class:

$$g(x) = p(y = 1 \mid x) = \sigma(s), \quad s = w^\top x + b, \qquad \sigma(s) = \frac{1}{1 + e^{-s}}.$$

Classification is obtained by thresholding (e.g. $g(x) \geq 0.5 \Rightarrow \hat{y} = 1$).

4.2.2 Bernoulli likelihood

For $y \in \{0, 1\}$, the Bernoulli model gives

$$p(y \mid x) = g(x)^y (1 - g(x))^{1-y}.$$

For i.i.d. samples:

$$\mathcal{L}(w,b) = \prod_{i=1}^{l} g(x_i)^{y_i}\big(1-g(x_i)\big)^{1-y_i}.$$

4.2.3 Cross-entropy loss (negative log-likelihood)

Maximizing $\mathcal{L}$ is equivalent to minimizing the NLL:

Binary Cross-Entropy / NLL

$$\mathcal{J}(w,b) = -\sum_{i=1}^{l}\Big[y_i \ln g(x_i) + (1-y_i)\ln\big(1-g(x_i)\big)\Big].$$

No closed-form solution $\Rightarrow$ use numerical optimization.

4.2.4 Gradients and convexity (what exams usually target)

For the above loss, the gradient has a simple structure:

$$\frac{\partial \mathcal{J}}{\partial w} = \sum_{i=1}^{l}\big(g(x_i)-y_i\big)x_i, \qquad \frac{\partial \mathcal{J}}{\partial b} = \sum_{i=1}^{l}\big(g(x_i)-y_i\big).$$

The objective is **convex** in (w,b), so any local optimum is global (the Hessian is positive semidefinite). Thus, getting stuck in a bad local minimum is not an issue for plain logistic regression (unlike neural networks).

4.3 Optimization in practice

4.3.1 Gradient descent and learning rate

Gradient descent iterates

$$w_{n+1} = w_n - \eta\,\frac{\partial \mathcal{J}}{\partial w}(w_n),$$

with learning rate $\eta > 0$. If η is too large, updates can diverge or oscillate; if too small, convergence is slow.

4.3.2 Momentum

Momentum accelerates gradient descent in flat regions and narrow valleys:

$$\Delta w_n = -\eta\,\frac{\partial \mathcal{J}}{\partial w} + \mu\,\Delta w_{n-1}, \qquad w_{n+1} = w_n + \Delta w_n,$$

with momentum parameter $0 \le \mu \le 1$.

4.3.3 Gradient checking (debugging tool)

To verify an analytic gradient implementation, compare to a numerical approximation via central differences:

$$\frac{\partial \mathcal{J}}{\partial w_i} \approx \frac{\mathcal{J}(w+\epsilon e_i) - \mathcal{J}(w-\epsilon e_i)}{2\epsilon},$$

with a typical choice $\epsilon = 10^{-4}$.

4.4 Multi-class logistic regression (softmax regression)

For K classes, define scores $s_k = w_k^\top x + b_k$ and probabilities

$$p(y = k \mid x) = \frac{e^{s_k}}{\sum_{j=1}^{K} e^{s_j}}.$$

With one-hot vectors $[y_i]_k \in \{0, 1\}$, the cross-entropy loss is

$$\mathcal{J}(\omega) = -\sum_{i=1}^{l} \sum_{k=1}^{K} [y_i]_k \ln p(y_i = k \mid x_i; \omega).$$

5

Artificial Neural Networks

5.1 Perceptron and linear separability

5.1.1 Perceptron as a linear threshold unit

For binary targets $y \in \{0, 1\}$, a perceptron implements a threshold decision:

$$g(x; w, \theta) = \begin{cases} 1, & w^\top x \geq \theta, \\ 0, & w^\top x < \theta. \end{cases}$$

The *decision boundary* is the hyperplane $w^\top x = \theta$ (linear in x). Even though the step function is non-smooth, the induced classifier is a **linear classifier** (linear separating hyperplane).

5.1.2 Perceptron learning algorithm (online updates)

The perceptron learning rule updates parameters only on mistakes. With learning rate $\eta > 0$ and samples (x_i, y_i):

- If $g(x_i; w, \theta) = 0$ but $y_i = 1$ (false negative), then

$$w := w + \eta x_i, \qquad \theta := \theta - \eta.$$

- If $g(x_i; w, \theta) = 1$ but $y_i = 0$ (false positive), then

$$w := w - \eta x_i, \qquad \theta := \theta + \eta.$$

- Otherwise: no update.

5.1.3 Linear separability, convergence, and XOR limitation

If the training set is **linearly separable**, the perceptron learning algorithm terminates and finds a separating hyperplane. The resulting solution is generally **not unique** (depends on initialization and data order). If the task is **not** linearly separable (classic example: XOR), the algorithm may fail to terminate.

This motivates **intermediate (hidden) layers** and yields the **multi-layer perceptron (MLP)**.

5.2 Feed-forward neural networks (MLPs / FNNs)

5.2.1 Architecture and forward pass notation

An MLP (often also called a fully connected feed-forward network, FNN) consists of layers $l = 0, 1, \ldots, L$, where $a^{[0]} = x$ is the input, and $a^{[L]} = \hat{y}$ is the output.

For each layer $l \geq 1$, define weights $W^{[l]}$, biases $b^{[l]}$, pre-activations $s^{[l]}$, and activations $a^{[l]}$:

$$s^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}, \qquad a^{[l]} = \phi^{[l]}(s^{[l]}),$$

where $\phi^{[l]}$ is an activation function (typically non-linear in hidden layers).

Intuition & Mental Model

Without activation functions, the composition of linear maps remains linear. Hence, an activation-free network cannot model non-linear functions.

5.2.2 Activation functions: sigmoid/tanh vs. ReLU/ELU

Classic activations include sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$ and $\tanh(z)$. Their derivatives can become small in saturation, contributing to **vanishing gradients** in deep nets.

Modern practice often uses ReLU:

$$\mathrm{ReLU}(x) = \max(0, x),$$

whose gradient is either 0 or 1 (helps against vanishing gradients). A known issue is **dying ReLUs** (units stuck at negative inputs, gradient 0).

ELU (exponential linear unit) is another option that keeps non-zero slope in the negative region and can improve optimization stability.

5.2.3 Loss functions (link to probabilistic assumptions)

In the course view, loss functions are tied to noise/likelihood assumptions:

- **Regression:** MSE corresponds to Gaussian noise; absolute error corresponds to Laplace noise; Huber can be used if noise is partly robust/heavy-tailed.
- **Classification:** cross-entropy maximizes the likelihood of correct classification; hinge loss enforces maximum margin (SVM).

5.2.4 Universal Approximation Theorem (exam-level statement)

A neural network with **at least one hidden layer** and a **non-linear activation function** can approximate any **continuous** function on a compact subset of $\mathbb{R}^d$, given sufficiently many hidden units. This is an *existence* statement; it does not guarantee easy training or good generalization.

5.3 Training: empirical risk, gradient descent, backpropagation

5.3.1 Empirical risk and gradient descent update

Let θ denote *all* learnable parameters (all weights and biases). Empirical risk:

$$R_{\text{emp}}(\theta) = \frac{1}{l} \sum_{n=1}^{l} L(y_n, g(x_n; \theta)).$$

Gradient descent updates parameters in the **negative** gradient direction:

Gradient Descent Parameter Update

$$\theta \leftarrow \theta - \eta \nabla_\theta R_{\text{emp}}(\theta).$$

Mini-batch SGD approximates the gradient on small batches, trading accuracy for speed and often improving generalization.

5.3.2 Backpropagation via delta errors and layer Jacobians

Backpropagation is the efficient evaluation of gradients via the chain rule; conceptually it is **reverse-mode automatic differentiation** for a scalar loss.

We use the standard layer notation

$$s^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}, \qquad a^{[l]} = \phi^{[l]}(s^{[l]}).$$

Define the **delta error** (vector of sensitivities) at layer l by

$$\delta^{[l]} := \frac{\partial L}{\partial s^{[l]}} \in \mathbb{R}^{n_l}.$$

With this definition, the parameter gradients at layer l take a very simple outer-product form:

$$\frac{\partial L}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^\top, \qquad \frac{\partial L}{\partial b^{[l]}} = \delta^{[l]}.$$

To propagate error signals backwards, note that

$$\delta^{[l-1]} = \frac{\partial L}{\partial s^{[l-1]}} = \left(\frac{\partial s^{[l]}}{\partial s^{[l-1]}} \right)^\top \delta^{[l]}.$$

For a fully connected layer, the relevant **single-layer Jacobian** mapping $s^{[l-1]} \mapsto s^{[l]}$ is

$$J^{[l]} := \frac{\partial s^{[l]}}{\partial s^{[l-1]}} = W^{[l]} \operatorname{diag}\Big((\phi^{[l-1]})'(s^{[l-1]}) \Big),$$

so the backward recursion is:

Backpropagation Backward Recursion

$$\delta^{[l-1]} = (J^{[l]})^\top \delta^{[l]} \qquad \text{equivalently} \qquad \delta^{[l-1]\top} = \delta^{[l]\top} J^{[l]}.$$

ReLU Derivative

$\mathrm{ReLU}(z) = \max(0, z)$ and

$$\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z < 0 \end{cases}$$

(at $z = 0$ one often uses 0 or a subgradient).

5.3.3 Delta error at the output layer (three important special cases)

For certain standard combinations of loss and output activation, the output delta error is simply "prediction minus label":

$$\delta^{[L]} = a^{[L]} - y,$$

for:

- squared loss with linear output,
- binary cross-entropy with sigmoid output,
- categorical cross-entropy with softmax output.

5.3.4 Backprop procedure (what you actually do)

A typical single-sample (or mini-batch) training step:

1. Forward pass; store all $s^{[l]}$ and $a^{[l]}$.
2. Compute $\delta^{[L]}$ at output units.
3. Propagate deltas backward using $\delta^{[l-1]\top} = \delta^{[l]\top} J^{[l]}$.
4. Compute gradients for all $W^{[l]}, b^{[l]}$.
5. Update parameters by a GD/SGD step.

5.4 Vanishing/exploding gradients (deep nets)

In deep networks, backprop repeatedly multiplies by Jacobians. Roughly,

$$\delta^{[0]} \approx \delta^{[L]} J^{[L]} J^{[L-1]} \dots J^{[1]}.$$

If Jacobian norms are < 1, gradients shrink exponentially (vanishing gradients). If norms are > 1, gradients blow up (exploding gradients).

Common mitigations covered in the course:

- suitable activations (ReLU/ELU) to avoid saturation,
- normalization (batch norm, layer/weight norm variants),
- clever initialization to keep Jacobians "well-conditioned",
- gating (for RNNs; see Unit 7),
- architectural tricks like skip connections (see ResNet in CNNs).

5.5 Deep learning tricks: initialization and regularization

5.5.1 Weight initialization (keep signal/gradients stable)

Bad initialization can already cause exploding/vanishing activations *before* learning starts.

Typical schemes:

- **Small Gaussian:** $w_{ij} \sim \mathcal{N}(0, 0.01)$ (can work for small nets, often fails for deep nets).
- **LeCun initialization:** for a layer with J input units,

$$w_{ij} \sim \mathcal{U}\left(-\frac{1}{\sqrt{J}}, \frac{1}{\sqrt{J}}\right),$$

  aiming for unit variance in activations.
- **Xavier/Glorot initialization:** for J inputs and I outputs,

$$w_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{I+J}}, \sqrt{\frac{6}{I+J}}\right),$$

  balancing variance across layers.
- **Orthogonal initialization:** initialize weights as random orthogonal matrices; helps keep Jacobians close to well-conditioned transformations.

5.5.2 Dropout (prevent co-adaptation)

Dropout is motivated by the fact that deep nets can overfit strongly. The idea is to randomly "drop out" units during training (different mask per iteration and per sample), so no unit can rely on specific other units being present.

Typical dropout rates mentioned in the lecture: 0.5 for hidden units and 0.2 for input units. Implementation note: outputs must be rescaled (either during training via "inverted dropout" or after training).

5.5.3 Batch normalization (BN) (stabilize learning)

BN normalizes activations to mean 0 and variance 1 (estimated over each mini-batch) and then learns an affine re-scaling:

Batch Normalization

For a mini-batch B with activations $a^{(1)}, \ldots, a^{(B)}$ of a neuron:

$$\mu_B = \frac{1}{B}\sum_{k=1}^{B} a^{(k)}, \qquad \sigma_B^2 = \frac{1}{B}\sum_{k=1}^{B}(a^{(k)} - \mu_B)^2,$$

$$\hat{a}^{(\xi)} = \frac{a^{(\xi)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad b^{(\xi)} = \gamma \hat{a}^{(\xi)} + \beta.$$

BN helps both optimization and regularization; variants include layer normalization and weight normalization.

5.5.4 Weight decay (L2 regularization)

Weight decay penalizes large weights, encouraging lower-complexity models. Conceptually:

$$R_{\text{emp}}(\theta) \;\rightsquigarrow\; R_{\text{emp}}(\theta) + \frac{\lambda}{2} \sum_{l} \left\| W^{[l]} \right\|_F^2 .$$

It is a standard regularizer to reduce overfitting (not underfitting).

5.5.5 Early stopping (validation-based regularization)

Early stopping monitors validation performance and stops training once validation error starts increasing, effectively acting as a regularizer (common in deep learning practice even if not formalized).

5.6 Double descent (optional intuition)

As model capacity increases, test error can exhibit "double descent": decreasing (bias reduction), then increasing near the interpolation threshold (variance peak), then decreasing again in an overparameterized regime. This is mainly a reminder that "more parameters" is not always monotone-bad for generalization.

6

Special Network Architectures

6.1 Convolutional Neural Networks (CNNs)

6.1.1 Why CNNs: image structure and local receptive fields

Images are very high-dimensional, but nearby pixels are highly correlated and basic patterns (edges, corners, textures) recur at many positions. CNNs exploit this by using **local receptive fields**: each neuron "looks" only at a small patch (kernel/filter), dramatically reducing parameters compared to full connectivity.

6.1.2 Convolution vs. cross-correlation and weight sharing

In deep learning, the operation called "convolution" is commonly implemented as **cross-correlation** (no kernel flip). With kernel weights $w_{i,j}$ (size $R \times R$) and input image $x_{a,b}$, the feature map pre-activation is:

$$s_{a,b} = \sum_{i=0}^{R-1} \sum_{j=0}^{R-1} w_{i,j}\, x_{a+i,b+j}.$$

Applying the **same** kernel weights at all spatial positions is called **weight sharing**. It is a key reason why CNNs generalize well to translated patterns.

6.1.3 Stride and padding

Striding uses step size $S > 1$ to downsample: we "skip" input positions while moving the kernel. This reduces computation but loses information; it also increases the receptive field through network depth.

Padding (often zero-padding) extends the input with P pixels around the borders. Padding can keep input and output sizes similar and prevents rapid shrinkage of feature maps.

CNN Output Size Formula

With input size A, kernel size R, padding P, stride S, the output size is

$$d = \frac{A - R + 2P}{S} + 1$$

(assuming the expression is an integer).

6.1.4 Pooling (subsampling)

Pooling reduces spatial resolution by aggregating over local windows (lossy downsampling). Common variants:

- **Average pooling:** average over a $k \times k$ field,
- **Max pooling:** maximum over a $k \times k$ field,
- **n-max pooling:** mean over the n largest values in a $k \times k$ field.

Exam Trap: Pooling and Receptive Field

Pooling does *not* decrease the receptive field through depth; downsampling layers (pooling/stride) typically *increase* the effective receptive field while reducing feature map resolution.

6.1.5 Inputs and outputs: channels and multiple kernels

CNN inputs often have multiple channels (e.g. RGB). A convolutional layer typically uses **multiple kernels**, each producing one output feature map (one output channel). Hence it is false that each convolutional layer always uses exactly one kernel.

Multi-Channel Convolution

For an input with channels $c = 1, \ldots, C_{\text{in}}$ (e.g. RGB) and one kernel with weights $w_{i,j,c}$:

$$s_{a,b} = \sum_{c=1}^{C_{\text{in}}} \sum_{i=0}^{R-1} \sum_{j=0}^{R-1} w_{i,j,c}\, x_{a+i,b+j,c}.$$

A convolutional layer typically uses C_{out} different kernels, producing C_{out} output feature maps.
Shape mnemonic: $(A \times A \times C_{\text{in}}) \mapsto (d \times d \times C_{\text{out}})$ with $d = \frac{A-R+2P}{S} + 1$.

6.1.6 Example architectures and transfer learning (high level)

Historically, CNNs achieved breakthroughs in ImageNet (ILSVRC). AlexNet (2012) popularized deep CNNs; later architectures (e.g. ResNet) introduced **skip connections**, improving optimization of very deep networks.

Transfer learning reuses pretrained kernels/weights (often from large datasets like ImageNet) and fine-tunes on a related target task. This can reduce data needs and training time.

6.1.7 Dataset augmentation (exam-relevant)

Dataset augmentation creates additional training examples via label-preserving transformations (e.g. crops, flips, small rotations, brightness/contrast jitter, noise). It is most useful when you have **few data** or **low diversity** and acts like regularization (improves robustness and reduces overfitting).

6.2 Recurrent Neural Networks (RNNs)

6.2.1 Sequence learning settings

Many problems involve sequences of variable length T, where each element has D features. A common categorization:

- **Sequence classification:** one label per sequence (e.g. sentiment of a sentence).
- **Segment classification:** one label per part of a sequence.
- **Temporal classification:** a label at each time step (sequence of labels; e.g. frame labeling).

6.2.2 RNN forward pass and weight sharing across time

An RNN maintains a hidden state over time. Using the lecture notation:

$$s(t) = W^\top x(t) + R^\top a(t-1), \qquad a(t) = f(s(t)), \qquad \hat{y}(t) = \phi(V^\top a(t)),$$

with input-to-hidden W, recurrent weights R, hidden-to-output V, and typically $a(0) = 0$. The defining property is **weight sharing** across time steps: the same (W, R, V) are reused for all t.

Turing Completeness: FNN vs. RNN

Feed-forward neural networks (FNNs) are typically not Turing-complete. Recurrent neural networks (RNNs) can be Turing-complete under suitable assumptions.

6.2.3 Training: unrolling and Backpropagation Through Time (BPTT)

Training uses backpropagation on the **unrolled** computational graph over time (BPTT). Complexity highlights:

- **BPTT:** time $\mathcal{O}(N^2T)$, memory $\mathcal{O}(NT)$ (store activations over time).
- **Truncated BPTT:** backprop only over a window of length τ; memory $\mathcal{O}(N\tau)$, but cannot capture dependencies beyond τ well.
- **RTRL:** alternative online method; time $\mathcal{O}(N^4T)$, memory $\mathcal{O}(N^3)$ (usually impractical).

6.2.4 Vanishing/exploding gradients in RNNs

RNNs are "deep in time", so gradients can vanish/explode similarly to deep feed-forward networks. A key stability condition (lecture form) is that we want:

$$\left\| R(t)\,\mathrm{diag}\big(f'(s(t-1))\big) \right\| \approx 1,$$

otherwise the influence of past states either decays too fast (vanishing) or blows up (exploding).

Common course-level mitigation ideas: careful initialization (often close to orthogonal for recurrent matrices), suitable activations, truncated BPTT, and **gating** (LSTM).

6.2.5 Long Short-Term Memory (LSTM): idea

LSTMs introduce a memory "cell" that acts like an **integrator** to preserve information over long time spans. The key mechanisms are **gates** that (i) decide what to remember, (ii) what to communicate to the output, and (iii) what to forget.

A key exam detail: the "constant error carousel" (CEC) is designed so that gradients can pass through time without vanishing, which is achieved via a **linear/identity** path (not by adding more non-linearities).

Standard LSTM Equations

With input x_t, hidden state h_{t-1}, cell state c_{t-1}, gates f_t, i_t, o_t:

$$f_t = \sigma(W_f x_t + R_f h_{t-1} + b_f), \quad i_t = \sigma(W_i x_t + R_i h_{t-1} + b_i),$$

$$\tilde{c}_t = \tanh(W_c x_t + R_c h_{t-1} + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,$$

$$o_t = \sigma(W_o x_t + R_o h_{t-1} + b_o), \quad h_t = o_t \odot \tanh(c_t).$$

6.2.6 Limitations and modern alternatives (very high level)

RNNs/LSTMs process sequences sequentially and can be harder to parallelize. This motivated a shift toward transformer-style attention models and, more recently, state space models (SSMs). The lecture notes mention this trend (e.g. "rise of transformers and SSMs", with examples like Mamba and xLSTM).

6.3 Attention and transformers (exam-level facts)

Attention computes weighted combinations of values based on similarity between queries and keys. In (scaled) dot-product form:

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Useful exam statements:

- Queries and keys share the *feature dimension* d_k but may have different sequence lengths (e.g. cross-attention).
- The attention output is invariant to simultaneously permuting keys and values in the same way (it only depends on their pairing).
- Standard dense self-attention is not exponential in sequence length; it is dominated by the $QK^\top$ computation (quadratic in T).